Mémo : Claude Code et GitHub Copilot

Formation IA pour les devs · commandes utiles, raccourcis et bonnes pratiques

Ce mémo rassemble l'essentiel à garder sous la main pendant les TP. Les interfaces évoluent vite : en cas d'écart avec votre version, la documentation officielle de l'outil fait foi.

1. Le prompt en 4 blocs

Bloc	Ce qu'on y met
Contexte	Projet, technologies et versions, fichiers concernés, situation actuelle
Tâche	Un objectif précis et unique, avec un verbe d'action ; demander un plan avant le code si la tâche est large
Contraintes	Style, dépendances autorisées, ce qu'il ne faut pas toucher, granularité des commits, lecture seule ou non
Critères de succès	Tests à faire passer, comportement attendu, format de la réponse, comment vérifier

2. Claude Code (CLI)

Démarrer et reprendre une session

Commande	Effet
`claude`	Démarre une session interactive dans le dossier courant (à la racine du projet)
`claude "question"`	Démarre avec une première demande
`claude -p "question"`	Mode non interactif : répond et rend la main (utile dans un script)
`claude -c`	Reprend la conversation la plus récente du dossier
`claude -r`	Choisit une conversation précédente à reprendre

Commandes slash intégrées

Commande	Effet
`/help`	Liste les commandes disponibles, y compris les vôtres
`/init`	Génère un `CLAUDE.md` initial à partir du projet
`/memory`	Affiche et édite les fichiers de mémoire (`CLAUDE.md`)
`/clear`	Efface la conversation : repartir d'un contexte propre
`/compact`	Résume la conversation pour libérer du contexte
`/context`	Montre l'occupation de la fenêtre de contexte
`/cost`	Affiche la consommation de la session
`/model`	Change de modèle
`/permissions`	Consulte et modifie les règles allow, ask, deny
`/agents`	Liste et crée les sous-agents
`/hooks`	Consulte la configuration des hooks

Commande	Effet
`/mcp`	Gère les serveurs MCP connectés
`/resume`	Reprend une conversation précédente
`/exit`	Quitte la session

Saisie et raccourcis

Touche ou préfixe	Effet
`@chemin/fichier`	Cite un fichier ou un dossier dans le prompt (complétion avec Tab)
`!commande`	Exécute une commande shell directement et ajoute sa sortie à la conversation
`Shift+Tab`	Change de mode : normal, acceptation automatique des éditions, plan
`Échap`	Interrompt l'action en cours
`Échap` deux fois	Revient sur un message précédent pour le modifier
`Ctrl+C`	Annule la saisie ; deux fois pour quitter

Fichiers de configuration du projet

Fichier	Rôle
`CLAUDE.md`	Mémoire du projet, chargée à chaque session (peut importer `@AGENTS.md`)
`.claude/commands/nom.md`	Commande slash personnalisée : `/nom` ; argument avec `$ARGUMENTS`
`.claude/agents/nom.md`	Sous-agent : nom, description, outils autorisés, consigne
`.claude/settings.json`	Permissions et hooks partagés avec l'équipe
`.claude/settings.local.json`	Réglages personnels, non versionnés
`.claude/hooks/`	Scripts appelés par les hooks

Permissions et hooks en bref

- Règles de permission dans `settings.json` : `allow` (autorisé), `ask` (demande), `deny` (interdit). `deny` l'emporte sur `ask`, qui l'emporte sur `allow`.
- Principe du moindre privilège : lecture et tests autorisés, écriture avec validation, secrets et commandes destructrices interdits.
- Hooks : `PreToolUse` (avant une action, peut la bloquer), `PostToolUse` (après : formatage, lint), `Stop` (avant de conclure : tests), `UserPromptSubmit`, `SessionStart`.
- Un hook reçoit un JSON sur l'entrée standard ; le code de sortie `2` bloque l'action et renvoie le message d'erreur à l'agent.
- Dans un hook `Stop`, testez `stop_hook_active` pour éviter une boucle infinie.

3. GitHub Copilot (VS Code)

Les trois modes de Chat

Mode	Usage
Ask	Poser des questions, explorer le code, obtenir un plan. Ne modifie pas vos fichiers

Mode	Usage
Edit	Modifier un ensemble de fichiers que vous choisissez ; vous validez chaque changement
Agent	Tâche multi-fichiers : l'agent lit, modifie, exécute des commandes et itère jusqu'à l'objectif

Contexte et commandes de Chat

Saisie	Effet
`#file:chemin`	Ajoute un fichier au contexte
`#codebase`	Laisse Copilot chercher dans tout le projet
`#selection`	Utilise le texte sélectionné dans l'éditeur
`#changes`	Utilise les modifications non committées
`@workspace`, `@terminal`, `@vscode`	Participants spécialisés : projet, terminal, VS Code
`/explain`	Explique le code sélectionné
`/fix`	Propose une correction du problème sélectionné
`/tests`	Génère des tests pour le code sélectionné
`/doc`	Génère de la documentation
`/clear` ou `/new`	Nouvelle conversation, contexte propre

Raccourcis courants (Windows et Linux)

Raccourci	Effet
`Ctrl+Alt+I`	Ouvre ou ferme la vue Chat
`Ctrl+I`	Discussion en ligne dans l'éditeur (modification ciblée)
`Tab`	Accepte la suggestion de complétion
`Échap`	Rejette la suggestion
`Alt+]` et `Alt+[`	Suggestion suivante et précédente

Sur macOS, remplacez `Ctrl` par `Cmd` pour la plupart des raccourcis.

Fichiers de personnalisation

Fichier	Rôle
`.github/copilot-instructions.md`	Instructions du projet, appliquées à chaque conversation
`.github/prompts/nom.prompt.md`	Prompt réutilisable, appelé par `/nom` dans Chat
`.github/agents/nom.agent.md`	Agent personnalisé avec sa consigne et ses outils
`AGENTS.md`	Règles partagées entre outils (prise en compte selon la version)

4. Équivalences entre les deux outils

Besoin	Claude Code	GitHub Copilot
Règles du projet	`CLAUDE.md` (importe `AGENTS.md`)	`.github/copilot-instructions.md`

Besoin	Claude Code	GitHub Copilot
Prompt réutilisable	`.claude/commands/` : `/review`	`.github/prompts/` : `/review`
Rôle spécialisé	`.claude/agents/` (sous-agents)	`.github/agents/` (agents personnalisés)
Lecture seule, plan	Mode plan (`Shift+Tab`)	Mode Ask
Tâche multi-fichiers	Session normale avec outils	Mode Agent
Citer un fichier	`@chemin`	`#file:chemin`
Repartir à zéro	`/clear`	`/clear` ou `/new`
Garde-fous automatiques	Hooks et permissions	Hook Git `pre-commit`, validation des commandes

5. Tips et bonnes pratiques

Bien demander

- Un objectif par prompt. Pour une tâche large : demander un plan, le relire, puis avancer par incréments.
- Citer les fichiers utiles plutôt que laisser l'outil deviner ; donner un fichier existant comme modèle de style.
- Écrire ce qu'il ne faut pas toucher. Les contraintes évitent les modifications opportunistes.
- Donner un retour factuel : « ce test échoue avec cette erreur » plutôt que « ça ne marche pas ».
- Demander des hypothèses classées avant de corriger un bug, et un test qui échoue avant la correction.

Garder le contrôle

- Relire chaque diff (`git diff`) avant d'accepter. Vous restez responsable du code livré.
- Un ticket, une branche, un commit par incrément : chaque étape devient un point de retour en arrière.
- Exiger la preuve : lancer les tests et afficher le résultat plutôt que croire « c'est fait ».
- Vérifier tout nom de paquet ou d'option inconnu avant de l'utiliser : les assistants peuvent en inventer.
- Un test généré qui passe ne prouve rien : cassez volontairement le code et vérifiez qu'un test devient rouge.

Gérer le contexte et le coût

- Donner le bon contexte, pas tout le contexte : une fenêtre trop pleine dégrade les réponses.
- Après trois tentatives qui tournent en rond, ouvrir une session propre avec un résumé de ce qui a échoué.
- Déléguer l'exploration volumineuse à un sous-agent : seule la synthèse revient dans la session principale.
- Surveiller la consommation (`/cost`, `/context`) lors de longues sessions ou de plusieurs agents.

Sécurité et confidentialité

- Ne jamais coller de secret, de clé ou de donnée personnelle dans un prompt ; ne jamais committer de `.env`.
- Respecter la politique de l'entreprise sur les données envoyées à un service externe.
- Relire tout hook ou script qui ne vient pas de vous : il s'exécute avec vos droits.
- Refuser les contournements : pas de `try/catch` ou de cas particulier qui masque un bug.

Quand l'outil dérive

Symptôme	Réaction
Il modifie trop de fichiers	Annuler hors périmètre, réaffirmer les fichiers autorisés, demander `git diff --stat`
Il affirme sans preuve	Demander d'exécuter la commande et de coller la sortie

Symptôme	Réaction
Il contourne le problème	Annuler le contournement, exiger l'explication de la cause
Il tourne en rond	Nouvelle session avec un contexte résumé et une contrainte supplémentaire

6. Avant d'accepter un changement

- Ai-je compris chaque ligne modifiée ?
- Le diff ne touche-t-il que les fichiers prévus ?
- `npm test` et `npm run lint` sont-ils verts, et l'ai-je vu moi-même ?
- Un test couvre-t-il le nouveau comportement, et peut-il échouer ?
- Aucun secret, aucune dépendance ajoutée sans raison ?
- Le message de commit décrit-il le changement en une ligne claire ?